

```

my_list = [3, 1, 4, 1, 5, 9, 2]
sorted_list = sorted(my_list)
print(f"Original list: {my_list}")
print(f"Sorted list (ascending): {sorted_list}")

sorted_desc = sorted(my_list, reverse=True)
print(f"Sorted list (descending): {sorted_desc}")

words = ["banana", "apple", "cherry"]
sorted_by_length = sorted(words, key=len)
print(f"Sorted by length: {sorted_by_length}")

```

```

Original list: [3, 1, 4, 1, 5, 9, 2]
Sorted list (ascending): [1, 1, 2, 3, 4, 5, 9]
Sorted list (descending): [9, 5, 4, 3, 2, 1, 1]
Sorted by length: ['apple', 'banana', 'cherry']

```

```

def custom_sort_key(item):
    # Example: sort by the sum of digits in a number
    return sum(int(digit) for digit in str(item))

numbers = [12, 5, 23, 10]
sorted_numbers = sorted(numbers, key=custom_sort_key)
print(sorted_numbers)

```

```
[10, 12, 5, 23]
```

```

my_list = [3, 1, 4, 1, 5, 9, 2]
my_list.sort()
print(f"Sorted list (in-place): {my_list}")

my_list.sort(reverse=True)
print(f"Sorted list (in-place, descending): {my_list}")

```

```

Sorted list (in-place): [1, 1, 2, 3, 4, 5, 9]
Sorted list (in-place, descending): [9, 5, 4, 3, 2, 1, 1]

```

```

data = [("apple", 5), ("banana", 2), ("cherry", 8)]
# Sort by the second element of each tuple (the number)
sorted_data = sorted(data, key=lambda item: item[1])
print(sorted_data)

```

```

words = ["apple", "banana", "cherry", "date"]
# Sort by the length of each string
sorted_words = sorted(words, key=len)
print(sorted_words)

```

```

[('banana', 2), ('apple', 5), ('cherry', 8)]
['date', 'apple', 'banana', 'cherry']

```

```

import bisect

my_list = [1, 3, 5, 7, 9]

# Find insertion points
index_right = bisect.bisect(my_list, 4)
index_left = bisect.bisect_left(my_list, 5)

print(f"Insertion point for 4 (right): {index_right}")
print(f"Insertion point for 5 (left): {index_left}")

# Insert elements while maintaining sorted order
bisect.insort(my_list, 4)
print(f"List after inserting 4: {my_list}")

bisect.insort_left(my_list, 5)
print(f"List after inserting 5 (left): {my_list}")

```

```

Insertion point for 4 (right): 2
Insertion point for 5 (left): 2
List after inserting 4: [1, 3, 4, 5, 7, 9]

```

```
List after inserting 5 (left): [1, 3, 4, 5, 5, 7, 9]
```

`bisect.bisect(a, x, lo=0, hi=len(a))`: This function (also aliased as `bisect.bisect_right`) returns the insertion point for `x` in `a` to maintain sorted order. If `x` is already present, the insertion point will be after (to the right of) any existing entries. The `lo` and `hi` parameters allow specifying a sub-range of the list to search within.

`bisect.bisect_left(a, x, lo=0, hi=len(a))`: Similar to `bisect.bisect`, but if `x` is already present, the insertion point will be before (to the left of) any existing entries.

`bisect.insort(a, x, lo=0, hi=len(a))`: This function (also aliased as `bisect.insort_right`) inserts `x` into `a` at the appropriate position to maintain sorted order. It uses `bisect.bisect` internally to find the insertion point.

`bisect.insort_left(a, x, lo=0, hi=len(a))`: Inserts `x` into `a` using `bisect.bisect_left` to determine the insertion point.

```
import bisect

my_list = [1, 3, 5, 7, 9, 11, 13]

# Using default lo and hi
insertion_point_full = bisect.bisect_left(my_list, 6)
print(f"Insertion point for 6 (full list): {insertion_point_full}")

# Using custom lo and hi (search only from index 2 to 5)
insertion_point_sublist = bisect.bisect_left(my_list, 6, lo=2, hi=6)
print(f"Insertion point for 6 (sublist from index 2 to 5): {insertion_point_sublist}")

Insertion point for 6 (full list): 3
Insertion point for 6 (sublist from index 2 to 5): 3
```

Since the `heapq` module in Python implements a min-heap by default, one can simulate a max-heap by negating the values before adding them to the heap and then negating them again when extracting the maximum value.

```
import heapq

# Initialize an empty heap
h = []

# Push items onto the heap
heapq.heappush(h, 3)
heapq.heappush(h, 1)
heapq.heappush(h, 2)
heapq.heappush(h, 5)

print(f"Heap after pushes: {h}")

# Pop the smallest item
smallest = heapq.heappop(h)
print(f"Smallest item popped: {smallest}, Heap after pop: {h}")

# Push and then pop
new_smallest = heapq.heappushpop(h, 0)
print(f"New smallest item after pushpop: {new_smallest}, Heap after pushpop: {h}")

# Find the n largest elements
data = [1, 8, 2, 9, 4, 7, 3, 6, 5]
largest_three = heapq.nlargest(3, data)
print(f"Three largest elements: {largest_three}")
smallest_three = heapq.nsmallest(3, data)
print(f"Three smallest elements: {smallest_three}")
```

```
Heap after pushes: [1, 3, 2, 5]
Smallest item popped: 1, Heap after pop: [2, 3, 5]
New smallest item after pushpop: 0, Heap after pushpop: [2, 3, 5]
Three largest elements: [9, 8, 7]
Three smallest elements: [1, 2, 3]
```

```
import heapq

max_heap = []
value_to_add = 10
heapq.heappush(max_heap, -value_to_add)
print(f"Heap after push: {max_heap}")
heapq.heappush(max_heap, -20)
```

```
print(f"Heap after push: {max_heap}")
heapq.heappush(max_heap, -5)
print(f"Heap after push: {max_heap}")
print(-heapq.heappop(max_heap))

Heap after push: [-10]
Heap after push: [-20, -10]
Heap after push: [-20, -10, -5]
20
```

The `heapq` module in Python provides an implementation of the heap queue algorithm, also known as the priority queue algorithm. It operates on regular Python lists, treating them as heaps. By default, it implements a min-heap, where the smallest element is always at index 0.

`heapq.heapify(x)`: Transforms a list `x` into a heap, in-place, in linear time. This is used to convert an existing list into a heap structure.

`heapq.heappush(heap, item)`: Pushes the item onto the heap, maintaining the heap invariant. This adds a new element while ensuring the heap property (smallest element at the root) is preserved.

`heapq.heappop(heap)`: Pops and returns the smallest item from the heap, maintaining the heap invariant. If the heap is empty, an `IndexError` is raised.

`heapq.heappushpop(heap, item)`: Pushes item onto the heap, then pops and returns the smallest item from the heap. This combined action is more efficient than separate calls to `heappush()` and `heappop()`.

`heapq.heapreplace(heap, item)`: This function pops the smallest item from the heap and then pushes the new item. This operation is also more efficient than a separate `heappop()` followed by `heappush()`.

Start coding or [generate](#) with AI.

```
import heapq

nums = [10, 20, 15, 30, 40]
max_heap_representation = [-n for n in nums]
heapq.heapify(max_heap_representation)
print(max_heap_representation) # Output: [-40, -30, -15, -10, -20]

[-40, -30, -15, -10, -20]
```

```
import heapq

# Create a regular, unordered list
my_list = [3, 1, 5, 7, 9, 2, 4, 6, 8]

print(f"Original list: {my_list}")

# Convert the list into a min-heap using heapq.heapify()
heapq.heapify(my_list)

print(f"Min-heap after heapify: {my_list}")

# You can now use other heapq functions, like heappop, to retrieve elements
smallest_element = heapq.heappop(my_list)
print(f"Popped smallest element: {smallest_element}")
print(f"Min-heap after heappop: {my_list}")

Original list: [3, 1, 5, 7, 9, 2, 4, 6, 8]
Min-heap after heapify: [1, 3, 2, 6, 9, 5, 4, 7, 8]
Popped smallest element: 1
Min-heap after heappop: [2, 3, 4, 6, 9, 5, 8, 7]
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.